AI for Business & Economic Research

From Chatbots to Agents

Mihail Velikov

Penn State — Smeal College of Business

AI Community of Practice in Research
April 6, 2026

Outline

1. Voices from the Field
2. Evolution of AI
3. How an Agent Works
4. Choosing Your Tools
5. Claude Code: Practical Advice
6. Use Cases Across the Research Workflow
7. Implications & Getting Started

1
Voices from the Field

Caveat Emptor

> "Every morning I wake up and think: do you know what we need more of? **Economists talking to us about AI** and what it's going to do to research."

— Andrew Baker, UC Berkeley Law

LinkedIn post

2024: Mass Production

> "This study represents an early exploration of AI's capabilities in producing academic finance research at scale. The tools and methods demonstrated here are in their infancy, and we expect significant advances in the coming years."

— Novy-Marx & Velikov (2026)

SSRN · GitHub

Early 2025: AI as Junior Co-Author

> "I had been repeatedly shocked by AI progress. I was using AI to prove theorems, vibe coding, and AI lit reviews in my daily life. Six months ago, I had thought each of these things is impossible. What will happen in the next six years?! Will my entire job be replaced by AI? **I have no idea.**"

— Andrew Chen, Federal Reserve Board

GitHub: Prompts-to-Paper

Full Year 2025: AI-First Research

> "If you look at my website, 2025 produced **a lot of working papers**. A few have been accepted for publication... A few others are R&R at better journals. I haven't yet broken through at a top-tier journal, but I did fail at a number."

— Joshua Gans, "Reflections on Vibe Researching" (2026) Substack

February 2026: Four Days to SSRN

> "I tried to write an academic paper in four weeks using AI... and **finished in four days.**"

— Vincent Gregoire, HEC Montreal

Blog post

2026: The 3-Hour Paper

> "I wrote a decent paper with AI. It took me about **3 hours** from start to finish, including an interactive choose-your-own-border RD adventure. It's a **what are we even doing here** kind of day."

— Paul Novosad, Dartmouth

X post

March 2026: Zero Human Involvement

> "Claude Code drives the **entire pipeline**: generating research ideas, writing R scripts, fetching data, running analysis, drafting LaTeX, and revising based on feedback. It operates as an agentic coding system — planning, executing, and iterating autonomously."

— Project APE

ape.socialcatalystlab.org

The arc: Years → Weeks → Days → Hours → **Zero**

Meanwhile, Outside Economics. . .

> "AI is now writing **much of the code** at Anthropic. . . We may be only **1–2 years away** from a point where the current generation of AI autonomously builds the next."

— Dario Amodei, CEO of Anthropic Dwarkesh Podcast

> "GPT-5.3-Codex is our first model that was **instrumental in creating itself**."

— OpenAI (2026)

> "An Erdős problem (#728) was solved **more or less autonomously by AI**. . . with the result not replicated in existing literature."

— Terence Tao, Fields Medalist Mastodon post

> In July 2025, AI earned **gold medals** at the International Mathematical Olympiad — three teams, three different approaches.

Google DeepMind blog

2
Evolution of AI

Three Paradigms of Generative AI

Traditional LLMs

Pattern-matching
"System 1" thinking
Language tasks
Now commoditized

Reasoning Models

Step-by-step
"System 2" thinking
Coding & math
PhD-level benchmarks

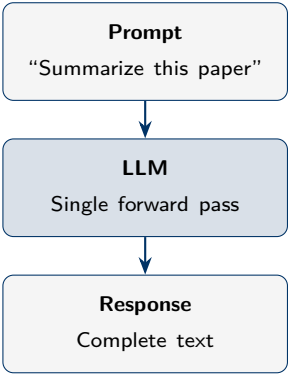
Agentic AI

Autonomous tool use
Planning & execution
Multi-step workflows
The current frontier

Amodei: AI models "substantially smarter than almost all humans at almost all tasks" on track for 2026–27.

Paradigm 1: Traditional LLMs

- Single-pass text generation ("System 1")
- Dominant for: writing, summarization, translation, brainstorming
- ChatGPT Instant 5.3, Claude Sonnet 4.6, Gemini 3 Flash, Llama 4, Qwen3



Key Takeaway

LLM capabilities are now **commoditized** — differentiation has shifted to reasoning and agentic capabilities.

Paradigm 2: Reasoning Models

- Chain-of-thought, step-by-step reasoning ("System 2")
- Trained via reinforcement learning to "think before answering"
- ChatGPT Thinking 5.4, Claude Opus 4.6, Gemini 3.1 Pro, Grok-4 Heavy

Benchmark	Best
GPQA (science)	94.3%
IMO (math)	Gold medal
SWE-Bench	∼75%

PhD experts score ∼65% on GPQA

PhD-Level Performance

These models now approach or exceed expert-level performance on scientific reasoning benchmarks.

GPQA Leaderboard (April 2026); Korinek (2025)

The Frontier Model Landscape (March 2026)

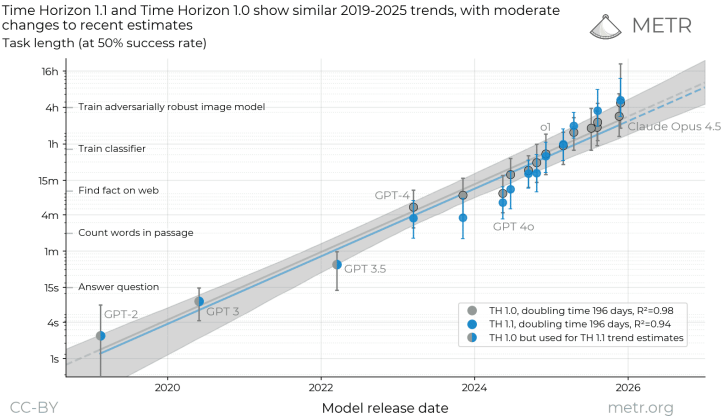
	OpenAI	Anthropic	Google
Fast / Traditional	Instant 5.3	Sonnet 4.6, Haiku 4.5	Gemini 3 Flash
Reasoning	Thinking 5.4, Pro	Opus 4.6 (Extended Thinking)	Gemini 3.1 Pro, Deep Think
Agentic / Coding	Codex (GPT-5.4)	Claude Code	Gemini CLI, Antigravity
GPQA	**93.2%** (Pro)	91.3%	**94.3%**

PhD experts score ∼65% on GPQA · All three labs achieved **IMO gold medals**

Also: **xAI** Grok-4 Heavy (88.4%) · **ByteDance** Seed 2.0 (88.9%) · **Alibaba** Qwen3.6 (90.4%)

GPQA Leaderboard (April 2026); Mollick (2026)

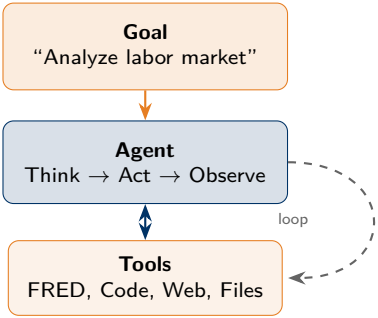
Exponential Progress



METR (2026); Mollick (2026); Svoronos (2026)

Paradigm 3: Agentic AI

- LLM + **tools** + **loop** + **goal**
- Agents *act* — they execute code, search the web, call APIs, read files
- Can plan multi-step strategies and adjust on the fly
- Spawn sub-agents for parallel work



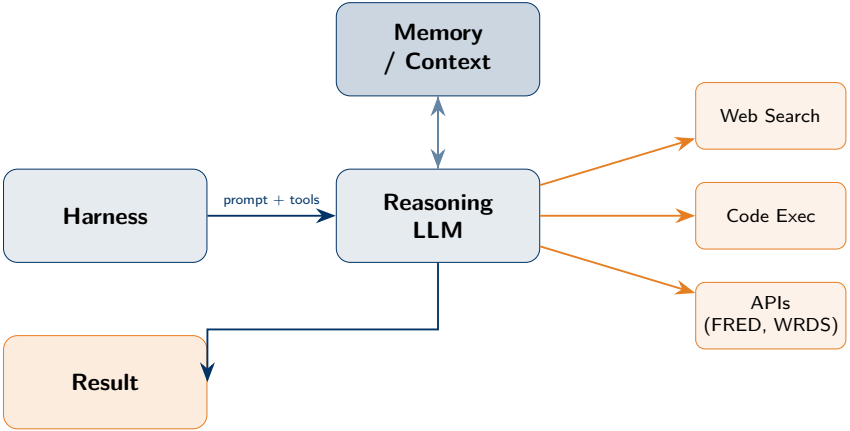
Key Shift

From "AI that says things" to **"AI that does things"**.

Korinek (2025); Svoronos (2026)

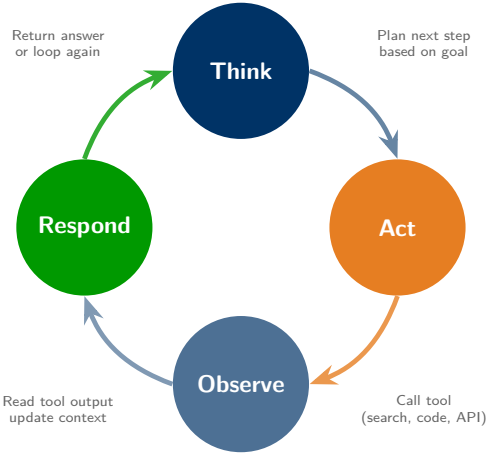
3
How an Agent Works

Agent Architecture



Korinek (2025)

The Think–Act–Observe Loop



FRED Agent: The Code

```
class FREDAgent:
  def __init__(self):
    self.fred = Fred(api_key=...)
    self.llm  = OpenAI(api_key=...)

  def think(self, question):
    prompt = "What␣FRED␣series␣code
␣␣␣␣␣␣answers:␣" + question
    # LLM returns JSON with code
    return llm_call(prompt)

  def act(self, series_code):
    return self.fred.get_series(
      series_code)

  def observe(self, data, units):
    return dict(value=data.iloc[-1],
                units=units)

  def respond(self, question, obs):
    return llm_call("Answer␣" +
      question + "␣using␣" + obs)

  def answer(self, question):  # harness
    code = self.think(question)
    data = self.act(code)
    obs  = self.observe(data)
    return self.respond(question,obs)
```

- ∼100 lines of Python
- The "intelligence" is in the **prompts** (lines 6–11, 22–23)
- Two LLM calls + one API call
- Fully **vibe-coded**: "A coding agent wrote this in under a minute" — Korinek

Key Insight

The agent decides *what* FRED series to fetch based on the question — it's not a script following predetermined steps.

Korinek (2025) — genaiforecon.org

FRED Agent: In Action

Question: "What is the US inflation rate?"

THINK: LLM selects series code → `FPCPITOTLZGUSA`

ACT: Calls `fred.get_series("FPCPITOTLZGUSA")` → 730 data points

OBSERVE: Latest value: 2.8% (March 2025), units: % change YoY

RESPOND: "U.S. inflation is 2.8% as of March 2025, down from. . . "

Two LLM calls + one API call = an "intelligent" data analyst.

Korinek (2025)

What Makes It an Agent?

- **Not a script**: it *decides* what to fetch based on the question
- **Modular**: think / act / observe / respond can each be extended
- **Composable**: same pattern works for WRDS, CRSP, Compustat, any API
- **Extensible**: add tools, add memory, add error recovery

From Simple to Sophisticated

1. Add more data sources (tools)
2. Add error handling & retries
3. Add sub-agents (parallel work)
4. Add memory across sessions
5. → Claude Code

> "What makes this code an 'agent' rather than a simple script is its autonomous, goal-directed behavior through multiple steps." — Korinek (2025)

Agents Are Powerful, Not Infallible

Failure Modes to Watch For

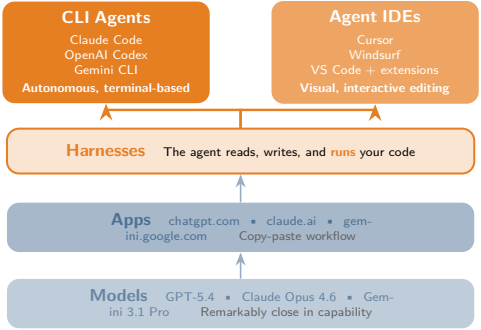
- **Hallucination**: agents confidently cite unreliable sources or fabricate data
- **Brittleness**: multi-step workflows can fail silently at any stage
- **Context loss**: long tasks degrade as context windows fill
- **Overconfidence**: formal-looking results that are subtly wrong
- **Security**: data leakage through API calls and tool access

> Human supervision remains essential. The question is not "can I trust the agent?" but "can I **verify** what the agent did?"

Korinek (2025); Gans (2026)

4
Choosing Your Tools

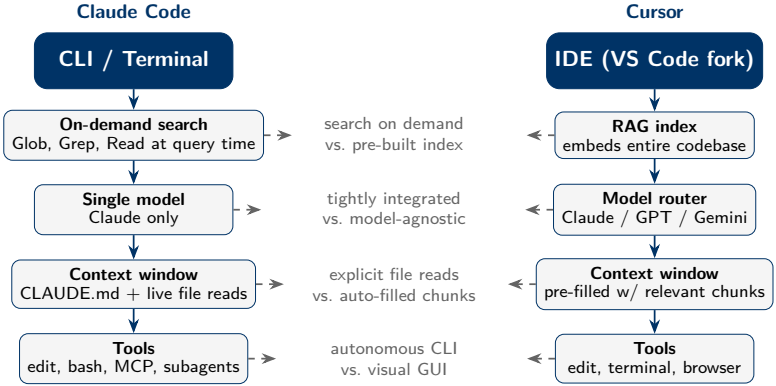
Models, Apps, and Harnesses



> The **harness** matters more than the model. The same model behaves very differently depending on what harness it operates in.

Mollick (2026), "A Guide to Which AI to Use in the Agentic Era"

Under the Hood: CLI Agents (Claude Code) vs. Agentic IDEs (Cursor)



Claude Code: finds code **when** you ask (explicit tool calls, always current)
Cursor: finds code **before** you ask (semantic search over embeddings)

CLI Agents: The Power Tools

	Claude Code	OpenAI Codex	Gemini CLI
Strengths	Most mature harness; compacting, skills, sub-agents, MCP	Strong reasoning; cloud-sandboxed execution	Google ecosystem
Price	$20/mo (Pro) $100/mo (Max 5x) $200/mo (Max 20x)	$20/mo (Plus) $200/mo (Pro)	$20/mo (AI Pro) $125/mo (AI Ultra)
Best for	Extended autonomous work	Budget-constrained users	

- Economics community split: 51% Claude Code, 18% Codex (Panjwani poll, March 2026)
- The landscape shifts fast — this is the current state, not the permanent answer
- Skills and project files are **portable** across tools

Mollick (2026); Panjwani (2026)

Agent-Native IDEs

Cursor | VS Code | Windsurf | Zed | Antigravity

- **Where they shine**: iterative editing with visual feedback — great for **teaching materials**, LaTeX papers, and code you want to review line-by-line
- **Model-agnostic**: Cursor lets you switch between Claude, GPT, Gemini mid-workflow
  Golub (Northwestern): "select Claude for coding, GPT for math-heavy proofs"
- **Overleaf** → GitHub → Cursor pipeline for collaborative LaTeX editing
- **The lines are blurring**: VS Code now has extensions for Claude Code, Codex, and Gemini — all CLI agents run inside an IDE
- **Pricing**: Cursor: Hobby (free) / Pro ($20/mo) / Ultra ($200/mo); VS Code free (extensions bring the agent)

> CLI agents and agentic IDEs **complement** each other — you can run Claude Code inside Cursor's terminal. Pick the right tool for the task, not the task for the tool.

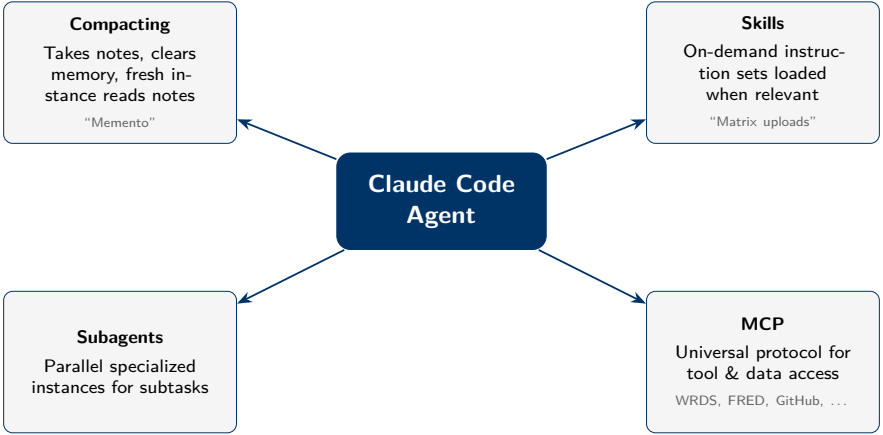
Golub (2025), Markus Academy; Mollick (2026)

Additional Useful Tools

Tool	Cost	What It Does
GitHub Pro	Free / $4	Copilot included; essential for agentic workflows
refine.ink	Free preview	AI referee reports; finds math errors & cross-paper inconsistencies
Whispr Flow	Free / $8	Voice-to-text dictation; built for **prompting by voice**
Obsidian	Free	Local markdown knowledge base; **wikilinks**, graph view, plugins
NotebookLM	Free / $20	Upload 300 papers; **paragraph-level citations**; generates podcasts, videos, slides
Elicit	Free / paid	AI literature search; extracts claims, methods, findings
Perplexity	Free / $20	Web search with inline citations; quick fact-checking

5
Claude Code: Practical Advice

The Agentic Harness: Claude Code



Mollick (2026), "Claude Code and What Comes Next"

CLAUDE.md — Teaching the Agent Your Rules

Project-Level CLAUDE.md

Lives in the repo root. Loaded **automatically** every session.

- Defines what this project *is* and how to work in it
- Rules, conventions, file structure — set once

```
# CLAUDE.md
## What This Project Is
Empirical asset pricing study.

## Rules
- All data pulled via WRDS API
- Use Python 3.12+, pandas, statsmodels
- Short variable names: ret, me, bm
- Compile LaTeX with latexmk
```

User-Level ~/.claude/CLAUDE.md

Personal preferences that follow you across **all projects**.

- Coding background, preferred stack, data sources
- Behavioral rules the agent always respects

```
# Preferences
- Do not assume -- confirm with me.
- Make atomic git commits as you go.
- Explain Python idioms (MATLAB background)

# Data
- WRDS API (wrds package)
- Core datasets: CRSP, Compustat
```

The agent's behavior is shaped by **persistent configuration**, not one-off prompts.

Plan Mode — Think Before You Build

How Plan Mode Works

1. You describe the task
2. Agent enters **read-only** exploration
3. Reads codebase, designs approach, identifies files to change
4. Writes a `PLAN.md` — persists even if context is compressed
5. Presents the plan for your approval
6. Only after you approve: editing begins

Why This Matters

- Prevents wasted effort on the wrong approach
- **Especially important for data pipelines** — the cost of re-running is high
- You stay in control; the agent does the legwork
- Plans survive context compression — the agent doesn't "forget" the approach mid-task

> Sant'Anna's "plan-first workflow": non-trivial tasks **always** start in plan mode. The agent can't edit files until you approve the plan.

Sant'Anna (2026), "My Claude Code Setup"

Skills — On-Demand Research Workflows

Slash Commands

Instruction files in `.claude/skills/` — loaded on demand when you type the command. Each defines a **multi-step workflow** the agent executes autonomously.

Research Skills (Sant'Anna)

- `/review-paper` — manuscript review with adversarial critic-fixer loop
- `/data-analysis` — end-to-end R/Python analysis pipeline
- `/lit-review` — literature synthesis
- `/research-ideation` — hypothesis generation
- `/interview-me` — formalize ideas through structured Q&A

10 Specialized Agents

- Proofreader
- Slide auditor
- Pedagogy reviewer
- R code reviewer
- TikZ critic
- Domain reviewer
- Adversarial QA pair (critic + fixer)
- Translator
- Verifier

Skills turn **recurring research workflows** into single commands. 22 slash commands + 18 context-aware rules. Adopted by 15+ research groups.

Sant'Anna (2026), "My Claude Code Setup"

Git Integration — Reproducibility by Default

Git-Aware from Birth

- Claude Code reads your **git history** to understand the project
- Creates branches, commits, and pull requests autonomously
- Every skill can end with a descriptive commit
- **Every change is tracked** — you can always revert
- Agentic IDEs (Cursor, Windsurf) have git integration **built in**

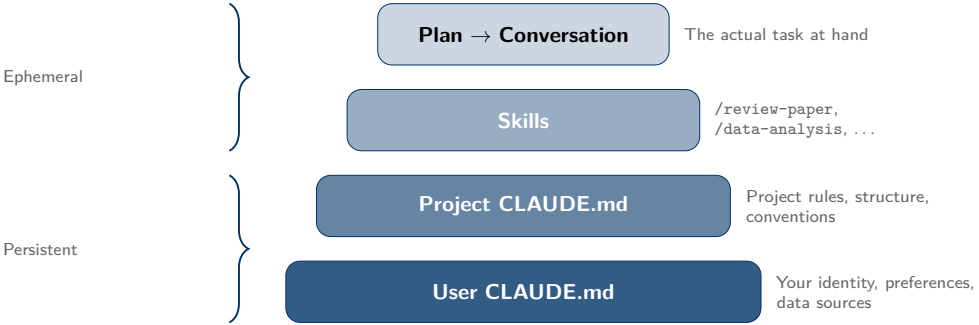
Configure Git in CLAUDE.md

```
## Git Rules
- Atomic commits w/ descriptive msgs
- Never commit raw data files
- Format: "Add: <description>"
- Branch for each new analysis
```

Why This Matters for Research

Every data transformation, every analysis step gets a commit message explaining *why*. Coauthors (and future you) can trace the full chain of reasoning. This is **computational reproducibility** without extra effort.

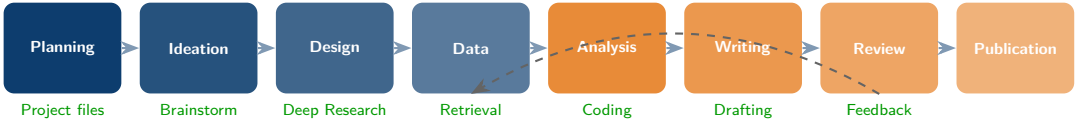
The Configuration Stack



Configure once, use everywhere. The agent improves as your **project infractructure** gets better — not as your prompts get longer.

6
Use Cases Across the Research Workflow

The Research Pipeline



AI Replicates

Execution skills: coding, data cleaning, drafting, formatting, literature search

Humans Provide

Judgment skills: research taste, identification strategy, institutional knowledge

Goldsmith-Pinkham (2025)

Ideation: The LLM Mirror

> "Drafted a blog post. Used an LLM to meticulously improve the argument over 4 hours. Wow, feeling great, it's so convincing! Fun idea let's ask it to argue the opposite. LLM **demolishes the entire argument** and convinces me that the opposite is in fact true. lol"

— Andrej Karpathy

LLMs as Devil's Advocate

LLMs are trained to argue *any* position persuasively. Use this deliberately: have the AI attack your own hypothesis before you invest months in a project.

Karpathy (2026), X post

Planning: AI Project Folders

The Problem

Every new AI conversation starts from scratch — you re-explain your preferences, constraints, and context every time.

The solution: project files

1. Dump everything about the domain
2. Have AI interview you (preferences, avoids, thresholds)
3. Synthesize into a canonical file (<500 words)

Examples

- **CLAUDE.md**: project instructions loaded automatically by coding agents
- **ChatGPT Projects**: persistent context across conversations
- **Research folders**: lit review prefs, journal filters, methodology standards

Files develop through use, not upfront design. Failures teach the rules.

Blattman (2026), "AI Project Folders"

Literature Review: Deep Research

What It Does

- Multi-agent architecture searches **hundreds of sources** in 5–30 minutes
- Compiles structured reports with citations
- Maps literature landscapes, identifies gaps
- "Potentially the single most valuable AI tool for faculty" — Blattman

Handle with Care

- May confidently cite **unreliable sources**
- Sometimes **fabricates** references entirely
- Best used as starting point, not final product
- Always verify citations against actual sources

Available in ChatGPT (Deep Research), Claude (research mode), Gemini (Deep Research).

Korinek (2025); Blattman (2026)

Data Collection: Claude Code WRDS Toolkit

Specialist Agents (Orlowski)

- `crsp-wrds-expert` — stock returns
- `optionmetrics-wrds-expert` — options
- `taq-wrds-expert` — high-frequency
- `wrds-query-orchestrator` — cross-database merges (link tables built in)

How It Works

- Describe data in **natural language**
- Agent picks the right database, schema, and link tables
- Skills pre-load schemas to avoid exploratory round-trips
- TAQ handled via SSH + SAS batch jobs on the WRDS grid

Key: Agents don't just fetch data — they **understand what to fetch**, which tables to join, and how to handle database-specific gotchas.

Orlowski (2026), Claude Code WRDS Toolkit

Data Collection: Empty Folder to Figure

Goldsmith-Pinkham — "How has the age distribution of homeowners changed over 50 years?"

A deliberately **vague** prompt to Claude Code in a brand-new directory.

What the agent did autonomously:

1. Tried FRED first — hit a dead end
2. Switched to Census Bureau website
3. Got 403 error — added user-agent header to bypass bot-blocking
4. Found *two* relevant tables with different tradeoffs
5. Decided to pull *both*
6. Parsed messy government Excel files
7. Produced two clean CSVs

Timeline: 6 minutes, 21 seconds

- Agent made *real decisions*
- Recovered from dead ends
- "It's exactly what you'd want an RA to do"
- Spawned a **sub-agent** for web research (context management)

Goldsmith-Pinkham (2026), Markus Academy

Data Collection: Iteration

The prompt trick:

> "Please make a figure in R. Follow the best practices of **Kieran Healy**."

- Claude translates this into: clean ggplot2, direct labeling, minimal chart junk, thoughtful color
- First version: broad age buckets (not what was needed)
- User iterates: "I want age on x-axis with different lines for each year"
- Agent re-fetches finer data, regenerates, auto-fixes overlapping labels

Key Lessons

- **Iteration** is fast and natural
- "What would take an hour of ggplot2 fiddling took three rounds of conversation"
- Agent is like a good RA: tries to make nice graphs, catches straightforward issues
- Human decides *what story matters*

Goldsmith-Pinkham (2026)

Data Collection: EDGAR Example

Before: Manual Pipeline

1. Write custom scraper
2. Parse HTML/XML filings
3. Handle edge cases manually
4. Debug encoding issues
5. Weeks of work

After: Agent-Assisted

1. Describe what you need in plain English
2. Agent writes scraper + parser
3. Agent handles edge cases
4. Agent tests and debugs
5. Hours, not weeks

AI replicates the **execution** (scraping, parsing) — human provides the **judgment** (what to extract, why it matters).

Goldsmith-Pinkham (2025)

Writing: Teaching AI Your Voice

Before: AI Default Style

- Generic, corporate tone
- "Furthermore..." "Delve into..."
- "Robust and compelling..."
- Sounds like *every* AI output

After: Voice File

- 5–10 writing samples analyzed
- AI generates style guide
- **Annotated examples** beat abstract rules
- Add a "ban list" of AI-isms

> The voice file approach: collect samples → analyze patterns → generate style spec → add annotated examples → maintain a ban list.

Blattman (2025), "Teaching AI Your Voice"

Review: AI Referee Reports

refine.ink

- Founded by **Benjamin Golub** (Northwestern) & **Yann Calvó López**
- Generates deep technical referee reports
- Detects **cross-paper inconsistencies** (e.g., definition on p. 3 used incompatibly on p. 42)
- Finds subtle **mathematical errors** (sign errors, logical gaps in proofs)

Why Now?

- AI-accelerated research makes quality control **more** important, not less
- Strongest on theory papers; works for empirical too
- Ironclad privacy: papers never shared or used for training
- Several journals piloting as complement to peer review

Free preview: 10 comments per paper. Upload TeX source for best results.

Golub (2025), Markus Academy Ep. 154

Full Pipeline: Vibe Research

Vincent Gregoire — "Investing in AGI"

- Idea from podcast → posted working paper in **4 days**
- Workflow: Claude outline → Claude Code overnight draft → multi-agent review loops → iterative refinement
- Tools: Quarto, Python, Git/GitHub
- Multi-agent review: Claude Code + Codex CLI + Gemini

- Caught 1 fabricated reference
- Caught 1 metadata error
- Human judgment still critical for idea selection, model steering

> "Faster, not smarter" — AI accelerated execution but didn't substitute for research taste.

Gregoire (2025)

Full Pipeline: Project APE

Automated Policy Evaluation

- David Yanagizawa-Drott's platform for **fully automated** empirical policy evaluation
- Input: a policy question
- Output: complete empirical paper with identification, estimation, and interpretation
- 592 papers generated as of March 2026

What it demonstrates:

- End-to-end research automation is here
- AI can handle data $\rightarrow$ estimation $\rightarrow$ writing

What it reveals:

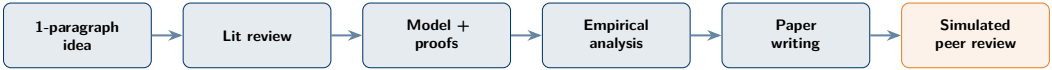
- 64.2% default to DiD
- Methodological monoculture risk
- More on this shortly...

Goldsmith-Pinkham (2026)

Full Pipeline: Automated Finance Papers

Lopez-Lira — Automated Finance Papers

Fully automated pipeline producing **submission-ready finance papers in ∼1 day**.



- Data: WRDS (CRSP, Compustat, TAQ), FRED
- Quality gates at each stage
- Handles theory, empirical, or both
- Target quality: ∼JFQA level
- Goal: published papers + meta-paper on the process itself

Lopez-Lira (2025)

Quality Control: Lessons from Vibe Research

Joshua Gans — 3 Pitfalls

1. **Subtle errors**: AI gets equilibrium concepts wrong — beyond simple math mistakes
2. **Idea inflation**: lower costs → pursuing weaker ideas
3. **Seductiveness**: confident results that are wrong

Guardrails

- **Cooling-off period** (1+ month)
- More peer feedback
- Explicit go/no-go decisions
- Human taste matters *more* than ever

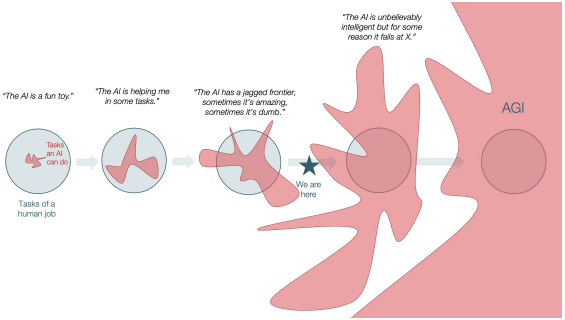
High-speed, low-human-input research did not produce high-quality output.

Gans (2026)

7
Implications & Getting Started

The Jagged Frontier

> "Shockingly capable at some hard things, embarrassingly bad at seemingly simple things." —
> Ethan Mollick, Wharton



Mollick et al. (2023)

Key Takeaways

1. **The shift is real** — academic researchers are already transforming their workflows
2. **Agents** > **chatbots** — autonomous tool use, planning, and multi-step execution
3. **The harness matters more than the model** — choose tools, not just AI brands
4. **Exponential progress** — task complexity doubling every 4–7 months
5. **The hard part remains human** — research taste, identification, institutional knowledge
6. **Start now, start small** — $20/mo gets you in the door

> "AI is a flashlight in the dark — the hard part is still knowing where to walk."
> — Goldsmith-Pinkham

Quick & Easy — Try This Today

Low-Risk, High-Reward First Projects

You don't need a greenfield project. Point an agent at something you **already have**.

1. **Reorganize a messy repo**

   Cunningham: "use a coding agent to clean up, refactor, and reorganize existing project directories, with Git for safety and rollback"

2. **Make slides from a paper**

   Feed it a PDF → get a Beamer or PowerPoint deck. Edit from there.

3. **Write a paper from a codebase**

   Point it at your analysis scripts → get a first draft describing data, methods, and results.

4. **Create a voice file**

   Give it 5–10 writing samples → it generates a style guide for future drafts.

5. **Get an AI referee report**

   Upload a working paper to refine.ink or use Sant'Anna's `/review-paper` skill.

6. **Translate code across languages**

   Stata → R → Python. Cross-language replication catches errors.

> All six work with a **$20/month** subscription. Start with what you already have — the agent meets you where you are.

Cunningham (2026); Blattman (2025); Panjwani (2026)

Economists to Follow on X on Agentic Coding

- Alex Imas: `@alexolegimas`
- Chris Blattman: `@cblatts`
- David Yanagizawa-Drott: `@YanagizawaD`
- Ethan Mollick: `@emollick`
- Kevin Bryan: `@Afinetheorem`
- Alexander Kustov: `@akoustov`
- Antonio Mele: `@antoniomele101`
- Anthony Lee Zhang: `@alz_zyd_`
- Aniket Panjwani: `@aniketapanjwani`
- Scott Cunningham: `@causalinf`
- Pedro Sant'Anna: `@pedrohcgs`
- Joshua Gans: `@joshgans`
- Jason Bell: `@j_jason_bell`
- Mihail Velikov: `@velikovmihail`

Adapted from Panjwani (2026). Also follow the AI developers: `@dylan522p`, `@birchwork`, `@vanhoornm`.

Resources

Blogs & Guides

- **Korinek**: genaiforecon.org
- **Mollick**: oneusefulthing.org
- **Blattman**: claudeblattman.com
- **Cunningham**: causalinf.substack.com
- **Goldsmith-Pinkham**: paulgp.substack.com
- **Panjwani**: aniketpanjwani.com
- **Sant'Anna**: Claude Code workflow

This Presentation's Wiki

- velikov-mihail.github.io/ai-econ-wiki

Live Demo: Claude Code

Let's see an agent work in real time.

Open Claude Code → give it a research-style prompt → watch Think–Act–Observe in action.

Thank you!

Mihail Velikov — velikov@psu.edu

> "Holy crap." — Chris Blattman, upon first using Claude Code